

RAPPORT DE STAGE D'INITIATION

Pour l'obtention du diplôme d'ingénieur

Conception et développement d'un assistant documentaire intelligent basé sur l'intelligence artificielle générative



Réalisé par : Jlaiel Manel

Encadré par (entreprise) : Khorcheffi Gaith

Organisme d'accueil : Talan Tunisie

Année universitaire : 2025/2026

Remerciements

Je tiens à exprimer mes sincères remerciements à toutes les personnes qui ont contribué au bon déroulement de mon stage et à sa réussite.

Je remercie tout particulièrement mon encadrant au sein de **Talan Tunisie**, pour son accompagnement, ses conseils et sa disponibilité tout au long de cette expérience.

Mes remerciements s'adressent également à l'ensemble de l'équipe de **Talan** avec laquelle j'ai eu l'occasion de travailler, pour leur accueil, leur disponibilité et leur soutien.

Enfin, je remercie toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet et m'ont permis d'enrichir mes connaissances et mon expérience professionnelle.

Table des matières

Introduction générale	1
Chapitre 1 : Cadre général du projet	3
1. Introduction.....	3
2. Présentation de l'organisme d'accueil	3
3. Cadre du projet.....	3
4. Présentation du projet.....	4
4.1. Description et critique de l'existant	4
4.2. Solutions existantes.....	4
4.2.1. Outils de type ChatPDF.....	4
4.2.2. NotebookLM (Google).....	4
4.2.3. GitHub Copilot Chat	5
4.3. Problématique	5
4.4. Solution proposée.....	5
5. Méthodologie de travail	6
6. Conclusion	6
Chapitre 2 : Analyse préliminaire et planification	7
1. Introduction.....	7
2. Présentation de l'équipe.....	7
3. Capture des besoins.....	7
3.1. Identification des acteurs.....	7
3.2. Backlog du produit.....	8
3.3. Besoins fonctionnels	8
3.3.1. Authentification et gestion des accès	8
3.3.2. Gestion des documents PDF	8
3.3.3. Interrogation des documents (RAG)	8
3.3.4. Analyse de projets de code.....	9
3.3.5. Gestion des rôles et permissions	9
3.4. Besoins non fonctionnels	9
4. Analyse préliminaire des besoins.....	10
4.1. Diagramme de cas d'utilisation global.....	10
4.2. Vue architecturale de l'application	10
4.2.1. Architecture physique	10
4.2.2. Architecture logique.....	11
4.3. Environnement matériel	12
4.4. Environnement logiciel	12
4.4.1. Outils de modélisation et de développement.....	12
4.4.2. Technologies utilisées	12
4.4.3. Langages de programmation	13
4.4.4. Outil de test	13

5. Planification des sprints	13
6. Conclusion	13
Chapitre 3 : Release 1 — Moteur RAG et microservice PDF	14
1. Introduction	14
2. Présentation de la release 1	14
3. Sprint 1 : Moteur RAG	14
3.1. Backlog du sprint	14
3.2. Conception	15
3.2.1. Diagramme de séquence	15
3.3. Réalisation	15
4. Sprint 2 : Microservice PDF-RAG (API FastAPI)	16
4.1. Backlog du sprint	16
4.2. Analyse	16
4.2.1. Diagramme de cas d'utilisation	16
4.2.2. Description textuelle du cas d'utilisation « Poser une question »	16
4.3. Conception	17
4.3.1. Diagramme de classes	17
4.3.2. Diagramme de séquence	17
4.4. Réalisation	18
5. Conclusion	19
Chapitre 4 : Release 2 — Code Explainer et Gateway	20
1. Introduction	20
2. Présentation de la release 2	20
3. Sprint 3 : Microservice Code Explainer	20
3.1. Backlog du sprint	20
3.2. Analyse	20
3.2.1. Diagramme de cas d'utilisation	20
3.2.2. Description textuelle du cas d'utilisation « Analyser un dépôt GitHub »	21
3.3. Conception	21
3.3.1. Diagramme de classes	21
3.3.2. Diagramme de séquence	22
3.4. Réalisation	22
4. Sprint 4 : Spring Cloud Gateway	23
4.1. Backlog du sprint	23
4.2. Conception	23
4.3. Réalisation	24
5. Conclusion	25
Chapitre 5 : Release 3 — Frontend et sécurité	26
1. Introduction	26
2. Présentation de la release 3	26
3. Sprint 5 : Interface Angular	26
3.1. Backlog du sprint	26

3.2. Analyse.....	26
3.3. Conception	26
3.4. Réalisation.....	27
4. Sprint 6 : Keycloak et gestion des rôles	28
4.1. Backlog du sprint	29
4.2. Analyse.....	29
4.3. Mécanisme de sécurité	29
4.4. Diagramme de séquence du scénario « S'authentifier »	30
4.5. Réalisation.....	30
5. Conclusion	31
Conclusion générale	32
Références bibliographiques	34

Table des figures

Figure 1 : Diagramme de cas d'utilisation global	10
Figure 2 : Architecture physique	11
Figure 3 : Architecture logique	11
Figure 4 : Diagramme de séquence — indexation d'un document	15
Figure 5 : Diagramme de cas d'utilisation du sprint 2	16
Figure 6 : Diagramme de classes du microservice PDF-RAG	17
Figure 7 : Diagramme de séquence — poser une question	18
Figure 8 : Interface Swagger UI du microservice PDF-RAG	19
Figure 9 : Diagramme de cas d'utilisation du sprint 3	21
Figure 10 : Diagramme de classes du microservice Code Explainer	21
Figure 11 : Diagramme de séquence — analyser un dépôt GitHub	22
Figure 12 : Résultat d'une analyse de dépôt GitHub	23
Figure 13 : Diagramme de séquence — routage et validation par la Gateway	24
Figure 14 : Architecture interne du frontend Angular	27
Figure 15 : Interface PDF	28
Figure 16 : Interface Code Explainer	28
Figure 17 : Diagramme de séquence — authentication via Keycloak	30
Figure 18 : Interface Login	31

Liste des tableaux

Tableau 1 : Répartition des rôles de l'équipe.....	7
Tableau 2 : Identification des acteurs.....	7
Tableau 3 : Backlog du produit	8
Tableau 4 : Environnement matériel	12
Tableau 5 : Outils de modélisation et de développement.....	12
Tableau 6 : Technologies utilisées	13
Tableau 7 : Langages de programmation	13
Tableau 8 : Planification des sprints	13
Tableau 9 : Backlog du sprint 1	14
Tableau 10 : Backlog du sprint 2	16
Tableau 11 : Backlog du sprint 3	20
Tableau 12 : Backlog du sprint 4	23
Tableau 13 : Backlog du sprint 5	26
Tableau 14 : Backlog du sprint 6	29

Introduction générale

Personne ne peut plus nier que l'intelligence artificielle constitue aujourd'hui une révolution fondamentale et innovante qui touche considérablement le monde professionnel. En effet, loin d'être un phénomène passager, l'intelligence artificielle générative s'impose désormais dans tous les aspects du travail quotidien. C'est ainsi qu'aujourd'hui, l'environnement des entreprises est devenu marqué par de profondes mutations. Ceci oblige notamment les entreprises informatiques à faire face à deux principaux défis : d'une part, la nécessité de traiter et d'exploiter rapidement un volume toujours croissant d'informations, qu'il s'agisse de documents internes ou de code source ; et d'autre part, l'exigence de proposer des outils sécurisés, fiables et parfaitement intégrés à l'écosystème technique déjà en place au sein de l'organisation.

Cette profonde mutation répond à un besoin vif pour n'importe quel type d'organisation : la capacité à retrouver rapidement une information précise dans un document volumineux, ainsi qu'à comprendre efficacement un projet de code existant, sont parmi les enjeux les plus déterminants pour la productivité des équipes techniques. C'est ainsi que nous considérons qu'une entreprise se doit d'exploiter les capacités de l'intelligence artificielle générative pour répondre à ces besoins, tout en maîtrisant pleinement la sécurité et la confidentialité de ses données.

C'est dans ce cadre que s'inscrit notre projet de stage d'initiation, intitulé « Conception et développement d'un assistant documentaire intelligent basé sur l'intelligence artificielle générative », que nous avons réalisé au sein de l'entreprise Talan. Cette application doit être sécurisée, fiable, évolutive et parfaitement intégrée à l'écosystème d'authentification de l'entreprise.

Pour retracer l'acheminement chronologique de notre travail, le présent rapport a été subdivisé en cinq chapitres. Le premier chapitre décrit le cadre général du projet : nous commençons par une présentation de l'organisme d'accueil, l'analyse et la critique de l'existant, et nous proposons par la suite la solution qui va être développée, ainsi que la méthodologie de conception et de développement adoptée. Le second chapitre s'articule autour de la phase de planification : nous y présentons l'équipe de travail, la capture des besoins, les fonctionnalités de notre futur système, le choix architectural, la configuration matérielle et logicielle, et enfin la planification des releases. Le troisième

chapitre est consacré à la première release, portant sur le moteur RAG et le microservice de gestion des documents PDF. Le quatrième chapitre est consacré à la deuxième release, où sont abordées l'analyse automatique de projets de code ainsi que la mise en place de la passerelle applicative. Le cinquième chapitre est consacré à la troisième et dernière release, portant sur le développement de l'interface utilisateur et la sécurisation complète de l'application. Le rapport s'achève par une conclusion générale présentant un bilan du travail réalisé ainsi que les perspectives futures de développement de l'application.

Chapitre 1 : Cadre général du projet

1. Introduction

Ce premier chapitre a pour objectif de situer notre projet dans son contexte général. Nous commençons par présenter l'organisme d'accueil, Talan, ainsi que ses domaines d'activité. Nous exposons ensuite le cadre dans lequel s'inscrit ce travail, avant de dresser un état des lieux critique de l'existant et de présenter quelques solutions comparables disponibles sur le marché. Cette analyse nous permet de dégager la problématique à laquelle notre projet répond, ainsi que la solution proposée. Nous terminons ce chapitre par la présentation de la méthodologie de travail adoptée tout au long de ce stage.

2. Présentation de l'organisme d'accueil

Fondée en 2002 à Paris par Mehdi Houas, Éric Benamou et Philippe Cassoulat, Talan est une entreprise de services numériques (ESN) et un cabinet de conseil en transformation digitale et agile des organisations. Le groupe accompagne ses clients dans leurs projets de transformation en s'appuyant sur l'innovation technologique, avec une attention particulière portée à l'intelligence artificielle, à la donnée et aux méthodes agiles.

Talan s'est rapidement développée à l'international, ouvrant des bureaux en Europe, en Amérique du Nord, en Afrique et en Asie. En 2007, le groupe inaugure son premier centre Nearshore à Tunis, donnant naissance à Talan Tunisie, aujourd'hui structurée comme le centre de développement Nearshore du groupe. Talan Tunisie regroupe plus de 500 ingénieurs spécialisés dans les nouvelles technologies, issus des grandes écoles d'ingénieurs tunisiennes et européennes, et intervient pour le compte de grands clients européens du groupe.

3. Cadre du projet

Le présent projet s'intitule « Conception et développement d'un assistant documentaire intelligent basé sur l'intelligence artificielle générative ». Ce travail s'inscrit dans le cadre d'un stage de fin d'études réalisé au sein de la société Talan Tunisie, du 01/07/2026 au 31/08/2026 en vue de l'obtention du diplôme d'ingénieur informatique.

4. Présentation du projet

4.1. Description et critique de l'existant

Au sein des grandes organisations telles que Talan, les collaborateurs sont amenés à consulter quotidiennement un volume important de documents : documentation de projets, supports de formation, comptes rendus, cahiers des charges. Ils interviennent également fréquemment sur du code source hérité ou développé par d'autres équipes. La recherche d'une information précise dans un document volumineux, tout comme la compréhension rapide d'un projet de code inconnu, représentent des tâches chronophages qui ralentissent la productivité des équipes.

Actuellement, ces tâches sont réalisées manuellement : lecture intégrale des documents pour y retrouver une information, ou exploration progressive d'un dépôt de code pour en comprendre l'architecture et les fonctionnalités exposées. Cette approche présente plusieurs limites : elle est lente, dépend fortement de l'expérience de la personne qui l'exécute, et ne permet pas une réutilisation aisée de la connaissance ainsi acquise.

4.2. Solutions existantes

Plusieurs outils basés sur l'intelligence artificielle générative existent aujourd'hui pour répondre partiellement à ce type de besoin. Nous présentons ci-après les plus pertinents pour notre contexte.

4.2.1. Outils de type ChatPDF

Des services comme ChatPDF permettent d'uploader un document PDF et de lui poser des questions en langage naturel. Ils reposent, comme notre solution, sur une architecture de type RAG.

- **Avantages** : Interface simple, prise en main immédiate, réponses généralement pertinentes sur un document unique.
- **Limites** : Solution SaaS externe : les documents sont envoyés et traités sur des serveurs tiers, ce qui pose un problème de confidentialité pour des documents internes à l'entreprise. Absence d'intégration avec le système d'authentification interne. Aucune fonctionnalité d'analyse de code.

4.2.2. NotebookLM (Google)

NotebookLM est un outil développé par Google permettant d'interroger simultanément plusieurs documents et d'en obtenir des synthèses structurées.

- **Avantages :** Bonne qualité de synthèse, gestion native de plusieurs documents à la fois.
- **Limites :** Outil grand public non conçu pour un usage en entreprise, hébergement des données hors du périmètre maîtrisé par l'entreprise, absence de gestion de rôles ou de droits d'accès différenciés, aucune fonctionnalité d'analyse de code.

4.2.3. GitHub Copilot Chat

GitHub Copilot Chat propose, entre autres fonctionnalités, une aide à la compréhension de code source directement dans l'éditeur ou sur la plateforme Git ;Hub.

- **Avantages :** Bonne qualité d'explication du code, intégration native à l'écosystème GitHub.
- **Limites :** Solution payante par utilisateur, dépendante du cloud Microsoft/GitHub, ne propose aucune fonctionnalité d'interrogation de documents PDF, pas d'intégration avec l'authentification interne de l'entreprise.

4.3. Problématique

Les outils présentés précédemment répondent chacun partiellement au besoin identifié, mais aucun ne réunit à la fois l'interrogation de documents PDF et l'analyse de projets de code au sein d'une même interface sécurisée et intégrée à l'écosystème d'authentification de l'entreprise. Se pose alors la question suivante : comment concevoir un assistant unique, sécurisé et évolutif, capable de répondre à des questions sur des documents internes tout en fournissant une vue d'ensemble rapide d'un projet de code, sans dépendre d'un service tiers externe non maîtrisé par l'entreprise ?

4.4. Solution proposée

Dans le cadre de ce stage, nous proposons de concevoir et de développer un assistant documentaire intelligent, reposant sur une architecture de microservices, et permettant notamment :

- **Interrogation de documents PDF :** Upload de documents PDF et formulation de questions en langage naturel, avec génération de réponses à partir du contenu réel des documents grâce à une architecture RAG (Retrieval-Augmented Generation).
- **Analyse de projets de code :** À partir d'un lien vers un dépôt GitHub, génération automatique d'un résumé structuré : objectif du projet, technologies utilisées, architecture, endpoints API détectés.

- **Authentification centralisée** : Intégration avec Keycloak, solution de gestion des identités, garantissant un accès sécurisé par authentification OAuth2/OpenID Connect.
- **Gestion des rôles** : Distinction entre utilisateurs standards et administrateurs, avec des permissions différenciées, notamment sur la visibilité et la suppression des documents.
- **Passerelle API centralisée** : Une Spring Cloud Gateway assurant le routage, la sécurité et la centralisation des accès aux différents microservices constituant la solution.

5. Méthodologie de travail

Face à la nature évolutive du projet, nous avons adopté une démarche de travail agile inspirée de la méthode SCRUM, qui consiste à découper le développement en itérations courtes appelées sprints, chacune donnant lieu à un incrément fonctionnel validé par l'encadrant, plutôt que de suivre une approche classique de type cycle en cascade où l'ensemble des besoins seraient figés dès le départ. Le stage se déroulant de manière individuelle, les rôles ont été adaptés au contexte : l'encadrant chez Talan a tenu le rôle de Product Owner, validant les priorités et les livrables à la fin de chaque sprint, tandis que l'étudiant a assumé à la fois les rôles de Scrum Master et d'équipe de développement. Cette approche, structurée autour de points de cadrage réguliers, de démonstrations fonctionnelles et de courtes rétrospectives, a permis d'avancer de façon progressive et de s'adapter rapidement aux ajustements nécessaires tout au long du stage.

6. Conclusion

Tout au long de ce premier chapitre, nous avons présenté le contexte général de notre projet, l'organisme d'accueil, ainsi que les limites des solutions existantes qui ont motivé la solution que nous proposons. Nous avons également justifié le choix de la méthodologie SCRUM, adaptée au contexte spécifique d'un stage individuel. Dans le chapitre suivant, nous abordons la phase d'analyse préliminaire et de planification de notre projet.

Chapitre 2 : Analyse préliminaire et planification

1. Introduction

Après avoir présenté le cadre général de notre projet, nous nous concentrons dans ce chapitre sur l'identification des besoins. Nous présentons dans un premier temps l'équipe du projet, les acteurs du système ainsi que le backlog de produit. Nous détaillons ensuite les besoins fonctionnels et non fonctionnels, avant de présenter le diagramme de cas d'utilisation global, l'architecture de la solution et l'environnement de travail. Nous terminons ce chapitre par la planification des sprints.

2. Présentation de l'équipe

Ce projet a été réalisé dans le cadre d'un stage individuel. Le tableau 1 détaille la répartition des rôles SCRUM adoptée, telle que présentée au chapitre précédent.

Rôle	Membre
Product Owner	Gaith Khorcheffi
Scrum Master / Équipe de développement	Manel Jlaiel

Tableau 1 : Répartition des rôles de l'équipe

3. Capture des besoins

La capture des besoins consiste à identifier les acteurs du système, à exposer le backlog du produit, puis à en déduire les besoins fonctionnels et non fonctionnels attendus.

3.1. Identification des acteurs

Notre système distingue deux acteurs principaux, dont les droits sont gérés via des rôles définis dans Keycloak et vérifiés à chaque requête par la passerelle applicative.

Acteur	Fonctionnalités
Utilisateur	Acteur authentifié pouvant uploader des documents PDF, interroger ses propres documents, analyser des dépôts GitHub, consulter son historique, et supprimer les documents qu'il a lui-même déposés.
Administrateur	Hérite de toutes les fonctionnalités de l'Utilisateur, et dispose en plus d'une visibilité sur l'ensemble des documents déposés par tous les utilisateurs, ainsi que du droit de les supprimer.

Tableau 2 : Identification des acteurs

3.2. Backlog du produit

Le backlog du produit, présenté dans le tableau 3, liste l'ensemble des user stories du projet, priorisées avec l'encadrant.

ID	User story	Priorité
US1	En tant qu'utilisateur, je peux m'authentifier à l'application via Keycloak.	Élevée
US2	En tant qu'utilisateur, je peux uploader un document PDF pour qu'il soit indexé.	Élevée
US3	En tant qu'utilisateur, je peux poser une question sur un document précis.	Élevée
US4	En tant qu'utilisateur, je peux poser une question sur l'ensemble de mes documents.	Moyenne
US5	En tant qu'utilisateur, je peux consulter la liste de mes documents indexés.	Élevée
US6	En tant qu'utilisateur, je peux supprimer un document que j'ai moi-même déposé.	Moyenne
US7	En tant qu'utilisateur, je peux analyser un dépôt GitHub à partir de son URL et obtenir un résumé structuré.	Élevée
US8	En tant qu'utilisateur, je peux consulter l'historique de mes conversations, séparé par document.	Moyenne
US9	En tant qu'utilisateur, je peux consulter l'historique des dépôts GitHub déjà analysés.	Faible
US10	En tant qu'administrateur, je peux consulter tous les documents déposés par l'ensemble des utilisateurs.	Moyenne
US11	En tant qu'administrateur, je peux supprimer n'importe quel document, quel que soit son propriétaire.	Moyenne
US12	En tant qu'utilisateur, mes requêtes vers les microservices doivent transiter par une passerelle unique et sécurisée.	Élevée
US13	En tant qu'utilisateur, je reçois un message d'erreur clair en cas d'échec (nom de fichier déjà existant, service indisponible).	Faible

Tableau 3 : Backlog du produit

3.3. Besoins fonctionnels

3.3.1. Authentification et gestion des accès

- S'authentifier via Keycloak (OAuth2 / OpenID Connect).
- Distinguer les droits d'un utilisateur standard de ceux d'un administrateur.

3.3.2. Gestion des documents PDF

- Uploader un document PDF et déclencher son indexation automatique.
- Consulter la liste de ses propres documents (ou de tous les documents pour un administrateur).
- Supprimer un document dont on est propriétaire (ou n'importe quel document pour un administrateur).

3.3.3. Interrogation des documents (RAG)

- Poser une question en langage naturel sur un document précis ou sur l'ensemble des documents.

- Recevoir une réponse générée à partir du contenu réel des documents, accompagnée des extraits utilisés.
- Conserver un historique de conversation persistant, séparé par document.

3.3.4. Analyse de projets de code

- Fournir l'URL d'un dépôt GitHub public et obtenir un résumé structuré (objectif, technologies, architecture).
- Visualiser les services et les endpoints API détectés automatiquement dans le code source.
- Consulter l'historique des dépôts déjà analysés sans avoir à relancer une analyse complète.

3.3.5. Gestion des rôles et permissions

- Restreindre certaines actions (suppression, visibilité globale) au rôle administrateur.
- Appliquer ces restrictions de façon centralisée au niveau de la passerelle applicative.

3.4. Besoins non fonctionnels

- **Sécurité** : Toutes les communications entre le client et les microservices transitent par une passerelle authentifiée ; aucun accès direct aux microservices n'est exposé.
- **Performance** : Le temps de réponse du système doit rester raisonnable malgré l'appel à un modèle de langage externe (Groq).
- **Fiabilité** : L'architecture en microservices indépendants permet à chaque brique de continuer à fonctionner même si une autre tombe en panne.
- **Maintenabilité** : Le code est organisé en modules clairement séparés (moteur RAG, API, sécurité, interface), facilitant sa compréhension et son évolution.
- **Utilisabilité** : L'interface propose un retour visuel clair à chaque action (indexation en cours, erreurs explicites, historique visible).
- **Portabilité** : Le recours à Docker pour Keycloak et à des microservices indépendants facilite le déploiement sur différents environnements.

4. Analyse préliminaire des besoins

Cette partie présente le diagramme de cas d'utilisation global ainsi que l'architecture générale retenue pour le projet, résultat de cette première analyse.

4.1. Diagramme de cas d'utilisation global

La figure 1 présente le diagramme de cas d'utilisation global, mettant en évidence les grandes fonctionnalités attendues du système ainsi que la relation de généralisation entre les deux acteurs : un administrateur héritant de toutes les capacités d'un utilisateur standard, avec des droits supplémentaires.

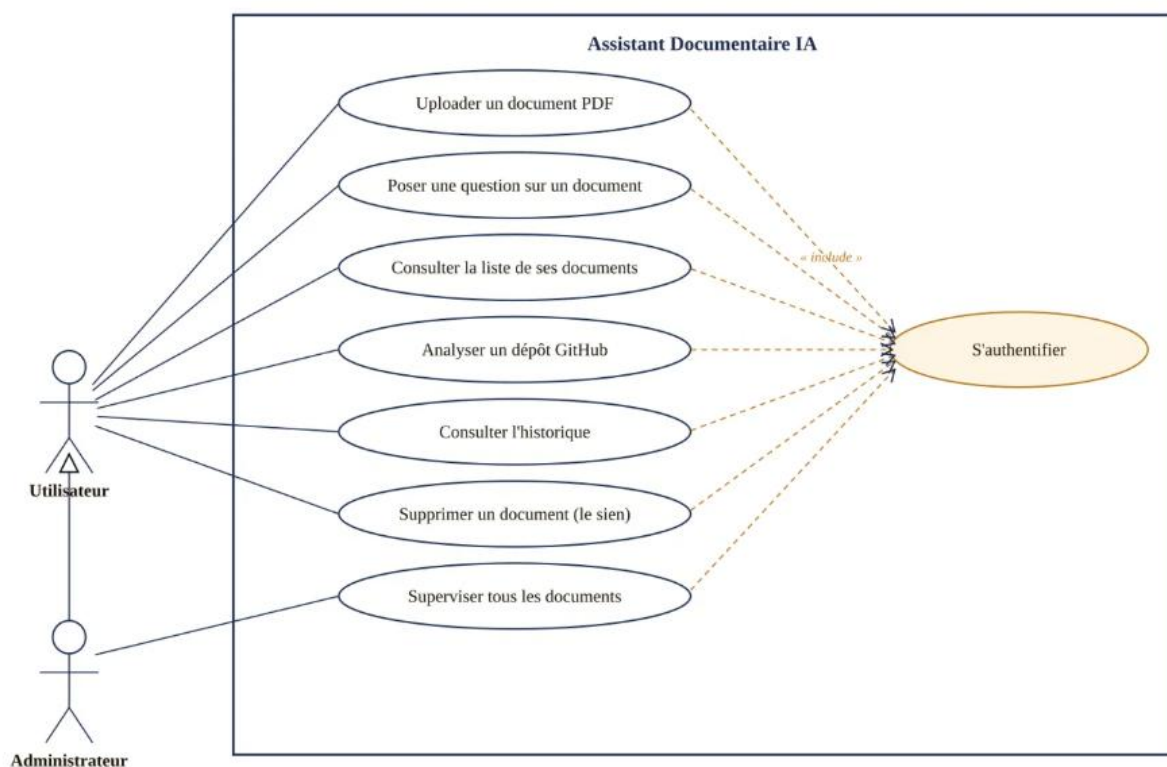


Figure 1 : Diagramme de cas d'utilisation global

4.2. Vue architecturale de l'application

Nous présentons dans cette partie les architectures physique et logique retenues pour notre application.

4.2.1. Architecture physique

Notre solution s'appuie sur une architecture à trois niveaux. Le client, exécuté dans le navigateur (application Angular), communique avec un serveur d'applications regroupant la passerelle Spring Cloud Gateway et les microservices FastAPI, qui

exposent les services métiers via des API REST. Ce serveur d'applications s'appuie à son tour sur un serveur de données (ChromaDB et fichiers PDF), ainsi que sur un serveur d'identité dédié (Keycloak), sollicité pour toute vérification d'authentification.

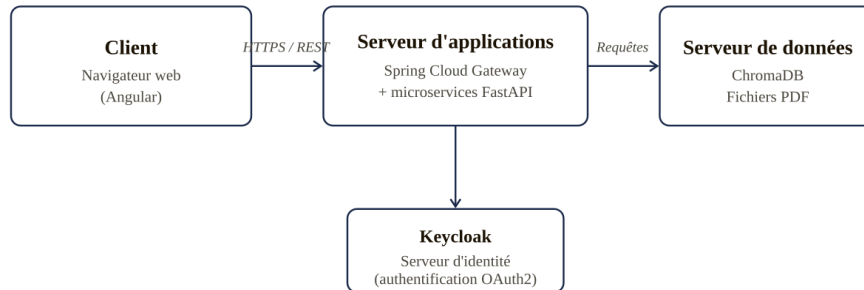


Figure 2 : Architecture physique

4.2.2. Architecture logique

Sur le plan logique, notre application repose sur une architecture orientée microservices. Ce choix garantit l'indépendance et l'évolutivité de chaque brique fonctionnelle : le microservice PDF-RAG et le microservice Code Explainer peuvent être développés, testés et déployés séparément, tout en étant accessibles depuis un point d'entrée unique.

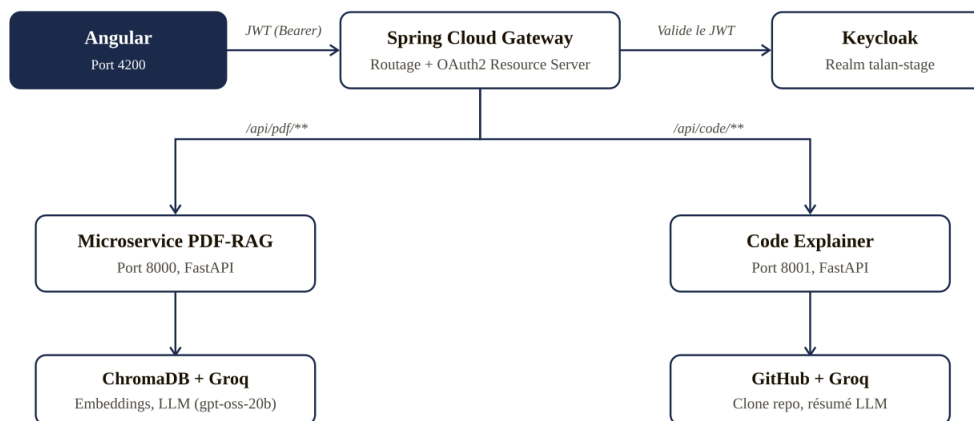


Figure 3 : Architecture logique

Le déroulement d'une requête suit les étapes suivantes :

- L'utilisateur s'authentifie auprès de Keycloak depuis l'application Angular, qui obtient en retour un token JWT.
- Ce token est automatiquement joint à chaque requête HTTP envoyée à la Spring Cloud Gateway.
- La Gateway vérifie la validité et la signature du token, puis extrait le nom d'utilisateur et ses rôles, avant de router la requête vers le microservice concerné selon le préfixe de l'URL.
- Le microservice traite la requête (indexation, recherche vectorielle, appel au LLM, ou clonage et analyse d'un dépôt) et renvoie une réponse à la Gateway, qui la transmet à Angular.

4.3. Environnement matériel

Ce projet a été réalisé à l'aide d'un ordinateur portable HP, dont les caractéristiques sont détaillées dans le tableau 4.

Caractéristique	PC : HP
Processeur	Intel CORE I5
RAM	8Go
Système d'exploitation	Windows 11

Tableau 4 : Environnement matériel

4.4. Environnement logiciel

4.4.1. Outils de modélisation et de développement

Logiciel	Description
Visual Studio Code	Éditeur de code utilisé pour le développement des microservices Python et de l'application Angular.
IntelliJ IDEA	Environnement de développement intégré utilisé pour la Spring Cloud Gateway (Java/Spring Boot[3]).
Docker Desktop	Utilisé pour exécuter Keycloak en local sous forme de conteneur.
Git / GitHub	Gestion de versions et hébergement du code source du projet.

Tableau 5 : Outils de modélisation et de développement

4.4.2. Technologies utilisées

Technologie	Description
FastAPI[1]	Framework Python utilisé pour exposer les microservices PDF-RAG et Code Explainier sous forme d'API REST.
Spring Cloud Gateway[4]	Framework Java assurant le routage, la sécurité et la centralisation des accès aux microservices.
Angular[2]	Framework front-end développé par Google, utilisé pour l'interface utilisateur de l'application.
ChromaDB[6]	Base de données vectorielle utilisée pour stocker et rechercher les embeddings des documents indexés.
Keycloak[5]	Solution open source de gestion des identités et des accès, utilisée pour l'authentification et la gestion des rôles.

Groq API[7]	Service d'inférence utilisé pour interroger le modèle de langage (LLM) générant les réponses et résumés.
-------------	--

Tableau 6 : Technologies utilisées

4.4.3. Langages de programmation

Langage	Description
Python	Utilisé pour le développement des microservices PDF-RAG et Code Explainer.
TypeScript	Langage utilisé pour le développement de l'interface Angular.
Java	Utilisé pour le développement de la Spring Cloud Gateway.

Tableau 7 : Langages de programmation

4.4.4. Outil de test

Afin de tester les microservices indépendamment de l'interface Angular, nous avons utilisé Swagger UI, généré automatiquement par FastAPI, permettant de documenter et de tester chaque endpoint directement depuis le navigateur.

5. Planification des sprints

Après avoir fixé notre backlog du produit, nous avons réparti l'ensemble des user stories sur trois releases, comme le présente le tableau 8.

Release	Sprint	User stories	Durée
Release 1 : Moteur RAG et microservice PDF	Sprint 1 : Moteur RAG	US2	2 semaines
	Sprint 2 : API FastAPI (PDF-RAG)	US3, US4, US5, US6	1 semaine
Release 2 : Code Explainer et Gateway	Sprint 3 : Microservice Code Explainer	US7, US9	2 semaines
	Sprint 4 : Spring Cloud Gateway	US12, US13	1 semaine
Release 3 : Frontend et sécurité	Sprint 5 : Interface Angular	US2 à US9 (UI)	1 semaine
	Sprint 6 : Keycloak et gestion des rôles	US1, US10, US11	1 semaine

Tableau 8 : Planification des sprints

6. Conclusion

Ce chapitre nous a permis d'identifier les acteurs de notre application, ainsi que les besoins fonctionnels et non fonctionnels associés, synthétisés à travers le backlog du produit et le diagramme de cas d'utilisation global. Nous avons ensuite présenté les architectures physique et logique retenues, ainsi que l'environnement matériel et logiciel utilisé. Enfin, nous avons exposé la planification des sprints qui structurera la suite de ce rapport. Le chapitre suivant sera consacré à la Release 1, portant sur le moteur RAG et le microservice de gestion des documents PDF.

Chapitre 3 : Release 1 — Moteur RAG et microservice PDF

1. Introduction

Après avoir établi le backlog du produit et réalisé l'analyse préliminaire des besoins, nous détaillons dans ce chapitre les deux sprints constituant la première release. Nous présentons pour chacun le backlog du sprint, l'analyse et la conception associées, ainsi qu'une présentation des choix de réalisation retenus.

2. Présentation de la release 1

Notre première release, intitulée « Moteur RAG et microservice PDF », comporte deux sprints :

- **Sprint 1 : Moteur RAG** : Mise en place du moteur d'indexation et de recherche sémantique, validé sous forme de script autonome.
- **Sprint 2 : Microservice PDF-RAG** : Exposition de ce moteur sous forme d'API REST via FastAPI, avec gestion complète des documents.

3. Sprint 1 : Moteur RAG

Avant d'exposer la moindre fonctionnalité via une interface, nous avons choisi de valider isolément le cœur algorithmique du système : la capacité à extraire, découper, vectoriser puis retrouver le contenu pertinent d'un document PDF. Cette étape a été menée sous forme de script Python autonome, exécuté en ligne de commande, sans interface graphique ni exposition réseau.

3.1. Backlog du sprint

ID	User story
US2	En tant que système, je dois pouvoir extraire le texte d'un PDF, le découper en fragments, calculer leurs représentations vectorielles et les stocker afin de permettre une recherche sémantique ultérieure.

Tableau 9 : Backlog du sprint 1

Ce sprint étant purement technique et ne donnant lieu à aucune interaction directe avec un acteur externe, nous ne présentons pas de diagramme de cas d'utilisation à ce stade : celui-ci sera introduit au sprint suivant, une fois ce moteur exposé sous forme d'API.

3.2. Conception

La figure 4 présente le diagramme de séquence du processus d'indexation d'un document, illustrant l'enchaînement des traitements internes du moteur RAG.

3.2.1. Diagramme de séquence

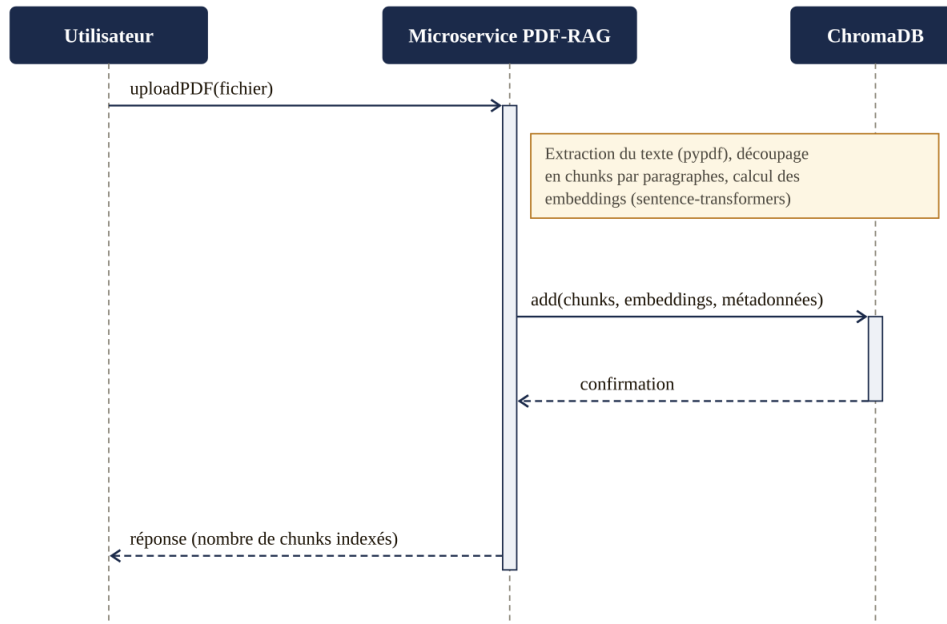


Figure 4 : Diagramme de séquence — indexation d'un document

3.3. Réalisation

Ce sprint n'ayant produit aucune interface graphique, sa validation s'est faite directement en ligne de commande. Plusieurs choix techniques ont été arrêtés à ce stade :

- **Découpage du texte** : Le texte est extrait à l'aide de la librairie pypdf[9], puis découpé par paragraphes plutôt que par un nombre fixe de caractères, afin de ne jamais interrompre une idée ou une définition en plein milieu d'un fragment.
- **Génération des embeddings** : Chaque fragment est vectorisé localement grâce au modèle sentence-transformers[8] (all-MiniLM-L6-v2), ce qui évite tout appel payant à une API externe pour cette étape.
- **Stockage vectoriel** : Les vecteurs obtenus sont stockés dans ChromaDB, une base de données vectorielle légère, choisie pour sa simplicité d'intégration et sa persistance locale.

Ce script a été testé sur plusieurs documents PDF réels, en vérifiant que les fragments retrouvés pour une question donnée correspondaient effectivement aux passages pertinents du document source, validant ainsi la fiabilité du moteur avant son exposition sous forme d'API au sprint suivant.

4. Sprint 2 : Microservice PDF-RAG (API FastAPI)

Ce second sprint consiste à exposer le moteur RAG validé précédemment sous forme d'un véritable microservice, accessible via des endpoints REST, et à y ajouter la génération de réponses en langage naturel ainsi que la gestion complète des documents.

4.1. Backlog du sprint

ID	User story
US3	En tant qu'utilisateur, je peux poser une question sur un document précis.
US4	En tant qu'utilisateur, je peux poser une question sur l'ensemble de mes documents.
US5	En tant qu'utilisateur, je peux consulter la liste de mes documents indexés.
US6	En tant qu'utilisateur, je peux supprimer un document que j'ai moi-même déposé.

Tableau 10 : Backlog du sprint 2

4.2. Analyse

4.2.1. Diagramme de cas d'utilisation

La figure 5 présente le diagramme de cas d'utilisation propre à ce sprint, limité au périmètre du microservice PDF-RAG.

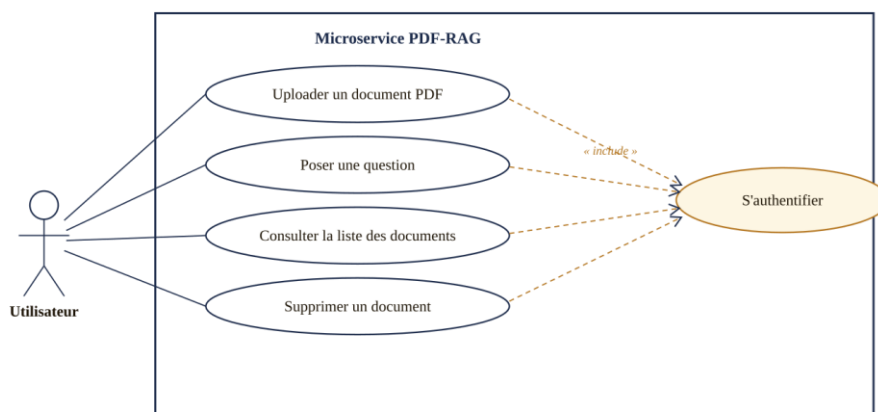


Figure 5 : Diagramme de cas d'utilisation du sprint 2

4.2.2. Description textuelle du cas d'utilisation « Poser une question »

Cas d'utilisation	Poser une question
Acteur	Utilisateur authentifié
Objectif	Obtenir une réponse fiable basée sur le contenu réel d'un ou plusieurs documents.
Résumé	L'utilisateur formule une question en langage naturel, que le système traite en retrouvant les passages pertinents avant de générer une réponse.
Préconditions	L'utilisateur est authentifié et possède au moins un document indexé.

Postconditions	Une réponse est affichée à l'utilisateur, accompagnée des extraits de documents utilisés pour la générer.
Scénario nominal	1. L'utilisateur sélectionne un document (ou l'option « Tous les documents »). 2. L'utilisateur saisit sa question et la soumet. 3. Le système recherche les fragments les plus pertinents dans la base vectorielle. 4. Le système transmet ces fragments et la question au modèle de langage. 5. Le système affiche la réponse générée ainsi que les extraits utilisés.
Exceptions	1. Si aucun passage pertinent n'est trouvé, le système répond explicitement qu'il ne trouve pas l'information, plutôt que d'inventer une réponse. 2. Si le microservice ou la passerelle est indisponible, un message d'erreur explicite est affiché.

4.3. Conception

4.3.1. Diagramme de classes

La figure 6 présente le diagramme de classes conceptuel de ce microservice. Un Document est composé de plusieurs Chunk (fragments indexés), chacun associé à son vecteur d'embedding. Lors d'une question, les Chunk les plus pertinents sont sélectionnés pour générer une ReponseGeneree.

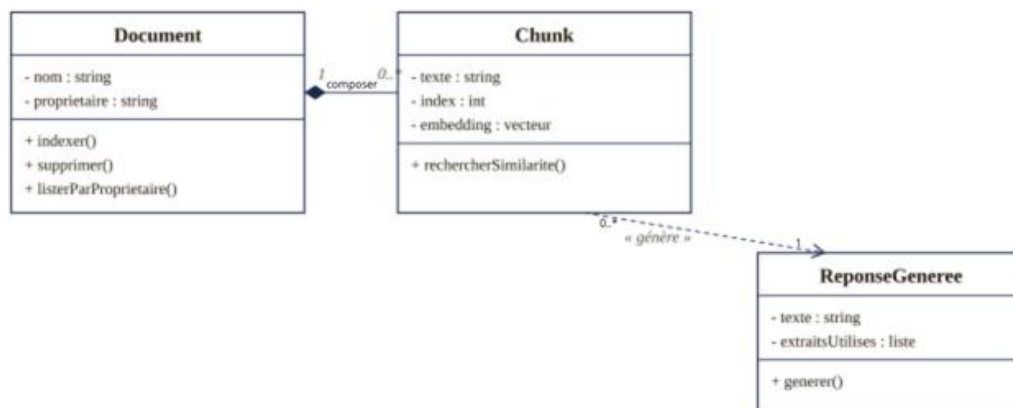


Figure 6 : Diagramme de classes du microservice PDF-RAG

4.3.2. Diagramme de séquence

La figure 7 présente le diagramme de séquence du scénario « Poser une question », depuis la requête initiale de l'utilisateur jusqu'à la génération de la réponse par le modèle de langage.

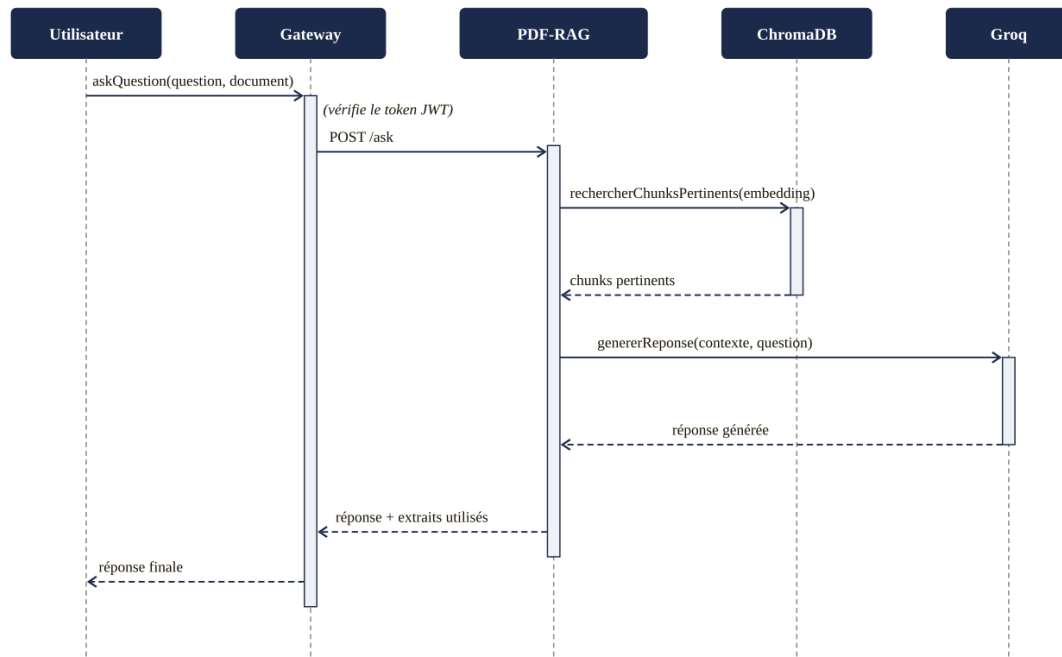


Figure 7 : Diagramme de séquence — poser une question

4.4. Réalisation

Le microservice a été développé avec FastAPI, qui génère automatiquement une documentation interactive (Swagger UI), utilisée pour tester chaque endpoint indépendamment de l'interface Angular, développée plus tard dans ce projet (voir chapitre 5).

- POST /upload — upload et indexation d'un document PDF.
- POST /ask — pose d'une question, avec filtrage optionnel sur un document précis.
- GET /documents — liste des documents indexés, filtrée selon le propriétaire et le rôle de l'utilisateur.
- DELETE /documents/{nom} — suppression d'un document, réservée à son propriétaire ou à un administrateur.

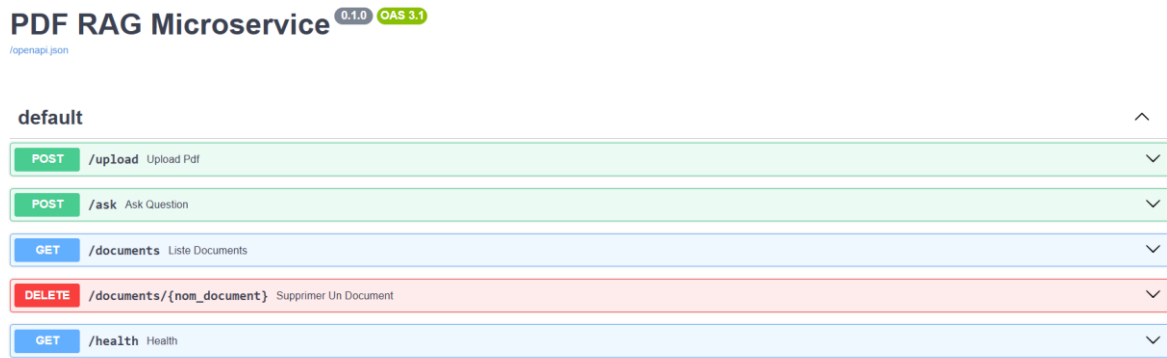


Figure 8 : Interface Swagger UI du microservice PDF-RAG

5. Conclusion

Ce chapitre nous a permis de présenter la première release de notre application, structurée en deux sprints. Le premier a permis de valider isolément le moteur RAG, avant son exposition, au second sprint, sous forme d'un microservice complet couvrant l'upload, l'interrogation, la consultation et la suppression de documents. Le chapitre suivant sera consacré à la release 2, portant sur le microservice d'analyse de code et la mise en place de la passerelle applicative.

Chapitre 4 : Release 2 — Code Explainer et Gateway

1. Introduction

En suivant le même principe que pour la première release, nous commençons par présenter cette nouvelle release, puis nous détaillons pour chaque sprint l'analyse, la conception et la réalisation associées.

2. Présentation de la release 2

Cette release est décomposée en deux sprints :

- **Sprint 3 : Microservice Code Explainer** : Mise en place du microservice d'analyse automatique de dépôts GitHub.
- **Sprint 4 : Spring Cloud Gateway** : Mise en place de la passerelle applicative centralisant l'accès aux deux microservices.

3. Sprint 3 : Microservice Code Explainer

Ce sprint vise à développer un second microservice, indépendant du premier, capable de cloner un dépôt GitHub et d'en produire un résumé structuré à l'aide d'un modèle de langage.

3.1. Backlog du sprint

ID	User story
US7	En tant qu'utilisateur, je peux analyser un dépôt GitHub à partir de son URL et obtenir un résumé structuré.
US9	En tant qu'utilisateur, je peux consulter l'historique des dépôts GitHub déjà analysés.

Tableau 11 : Backlog du sprint 3

3.2. Analyse

3.2.1. Diagramme de cas d'utilisation

La figure 9 présente le diagramme de cas d'utilisation de ce sprint.

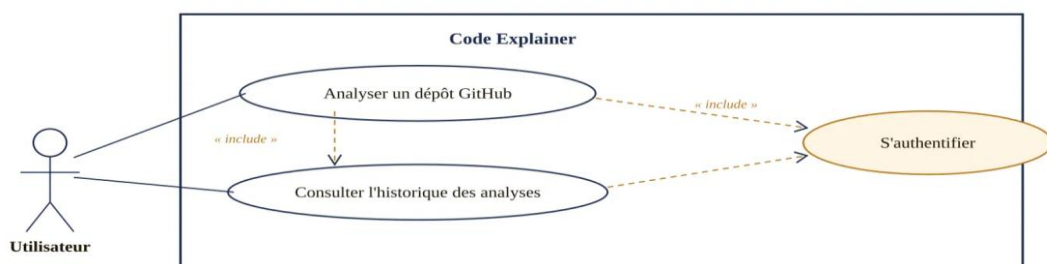


Figure 9 : Diagramme de cas d'utilisation du sprint 3

3.2.2. Description textuelle du cas d'utilisation « Analyser un dépôt GitHub »

Cas d'utilisation	Analyser un dépôt GitHub
Acteur	Utilisateur authentifié
Objectif	Obtenir rapidement une vue d'ensemble d'un projet de code sans l'explorer manuellement.
Résumé	L'utilisateur fournit l'URL d'un dépôt public, que le système clone, scanne et résume automatiquement.
Préconditions	L'utilisateur est authentifié et fournit une URL de dépôt GitHub public valide.
Postconditions	Un résumé structuré est affiché, incluant les technologies, services et endpoints détectés.
Scénario nominal	1. L'utilisateur saisit l'URL du dépôt à analyser et valide. 2. Le système clone temporairement le dépôt. 3. Le système lit le README, détecte les fichiers de configuration et scanne le code à la recherche d'endpoints API. 4. Le système transmet ces éléments au modèle de langage pour générer un résumé structuré. 5. Le système affiche le résumé, la liste des services et des endpoints détectés, puis supprime le dépôt cloné.
Exceptions	1. Si l'URL est invalide ou le dépôt inaccessible, un message d'erreur explicite est renvoyé. 2. Si aucun README n'est trouvé ou qu'aucun endpoint n'est détecté, le résumé le mentionne explicitement plutôt que d'inventer une information.

3.3. Conception

3.3.1. Diagramme de classes

La figure 10 présente le diagramme de classes conceptuel de ce microservice. Un RepoGitHub, une fois analysé, est associé à plusieurs ServiceDetecte et EndpointDetecte, ainsi qu'à un unique ResumeAnalyse généré par le modèle de langage.

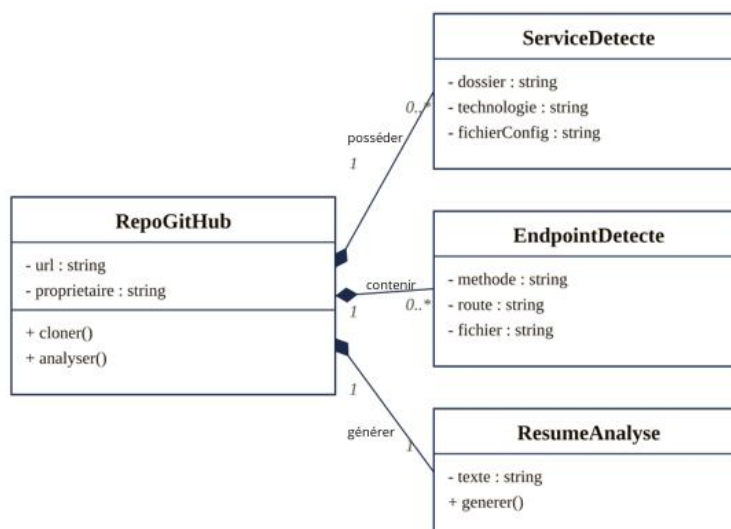


Figure 10 : Diagramme de classes du microservice Code Explainer

3.3.2. Diagramme de séquence

La figure 11 présente le diagramme de séquence du scénario « Analyser un dépôt GitHub ».

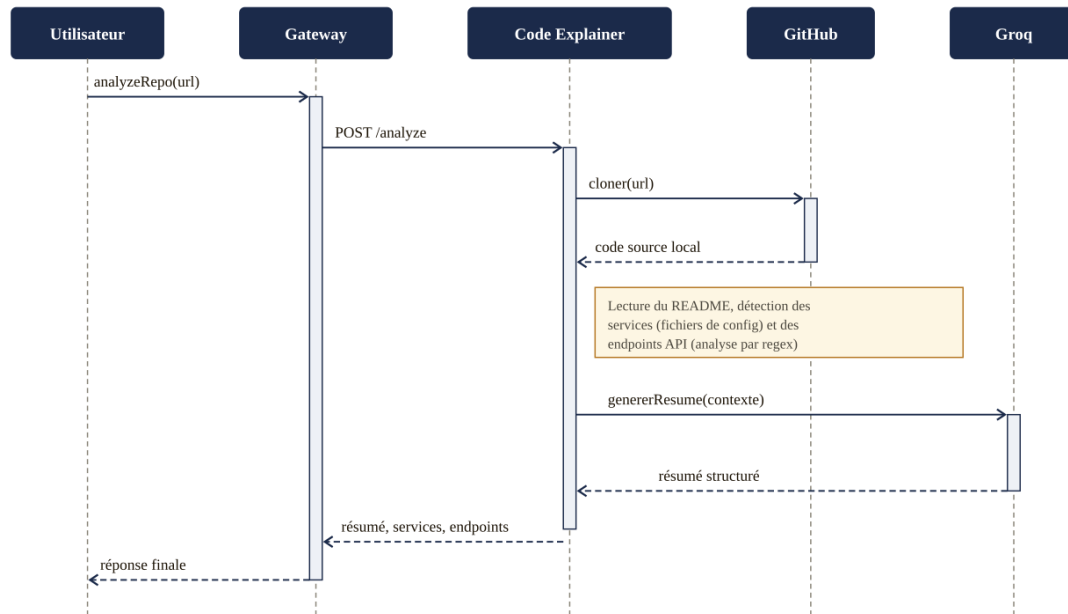


Figure 11 : Diagramme de séquence — analyser un dépôt GitHub

3.4. Réalisation

Le microservice expose deux endpoints principaux : POST /analyze, qui déclenche le clonage et l'analyse d'un dépôt, et GET /health, utilisé pour vérifier son bon fonctionnement.

Plusieurs ajustements ont été nécessaires au cours de ce sprint pour fiabiliser la détection. En particulier, la détection du README a dû être élargie pour couvrir les fichiers sans extension (certains dépôts utilisent un fichier nommé simplement README, sans suffixe .md), et la détection des endpoints a nécessité une déduplication, certains frameworks pouvant être détectés par plusieurs expressions régulières simultanément pour une même route.

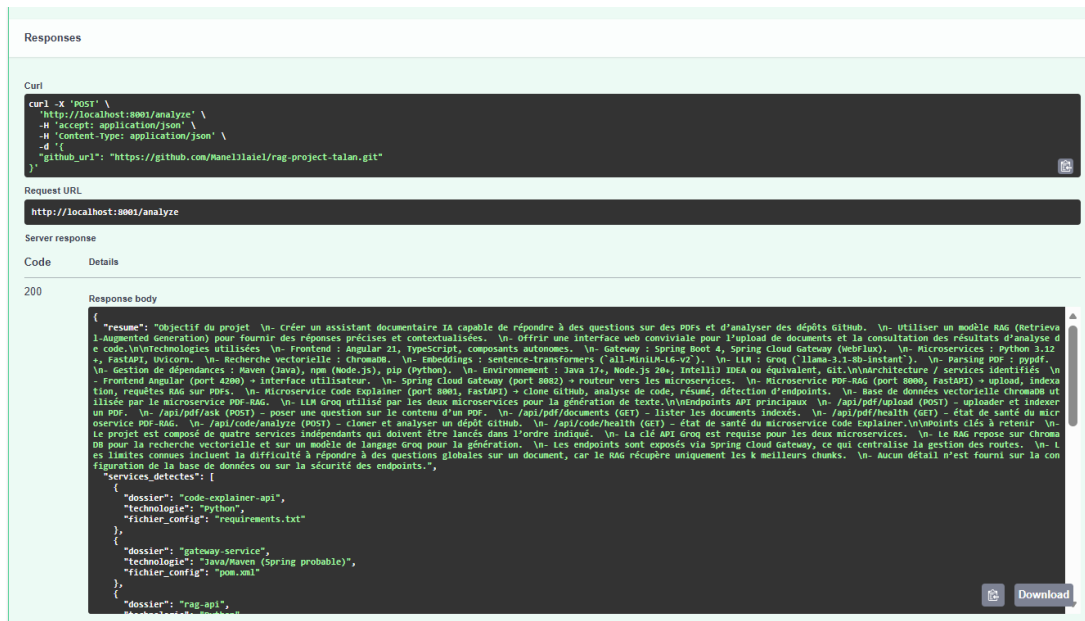


Figure 11 : Résultat d'une analyse de dépôt GitHub

4. Sprint 4 : Spring Cloud Gateway

Ce sprint est consacré à la mise en place d'une passerelle applicative unique, centralisant l'accès aux deux microservices développés précédemment.

4.1. Backlog du sprint

ID	User story
US12	En tant qu'utilisateur, mes requêtes vers les microservices doivent transiter par une passerelle unique et sécurisée.
US13	En tant qu'utilisateur, je reçois un message d'erreur clair en cas d'échec (nom de fichier déjà existant, service indisponible).

Tableau 12 : Backlog du sprint 4

La Gateway étant un composant d'infrastructure, transparent pour l'utilisateur final, ce sprint ne donne pas lieu à un diagramme de cas d'utilisation dédié : aucun nouvel acteur ni nouvelle action visible n'est introduit du point de vue de l'utilisateur. Nous présentons directement la conception de son fonctionnement interne.

4.2. Conception

La figure 13 illustre le rôle central de la Gateway : chaque requête entrante est d'abord authentifiée auprès de Keycloak, avant d'être routée vers le microservice concerné selon le préfixe de son URL.

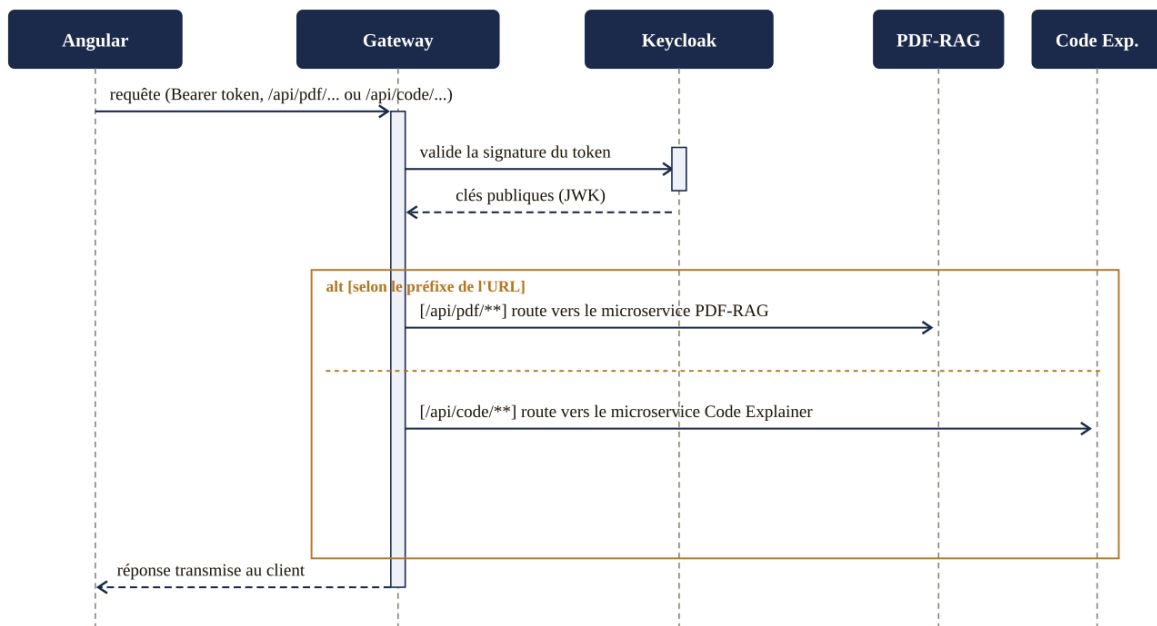


Figure 13 : Diagramme de séquence — routage et validation par la Gateway

4.3. Réalisation

La Gateway a été développée avec Spring Cloud Gateway, dans sa variante réactive (WebFlux). Sa configuration définit deux routes principales :

- **/api/pdf/**** : Toute requête commençant par ce préfixe est redirigée vers le microservice PDF-RAG (port 8000), après retrait de ce préfixe.
- **/api/code/**** : Toute requête commençant par ce préfixe est redirigée vers le microservice Code Explainer (port 8001).

Une configuration CORS dédiée a également été mise en place pour autoriser les échanges avec le frontend Angular, ainsi qu'un endpoint /health exposé via Spring Boot Actuator, permettant de vérifier rapidement la disponibilité de la passerelle.

Extrait du fichier de configuration application.yml de la Gateway, illustrant la définition des deux routes principales :

```

spring:
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: pdf-rag-service
              uri: http://localhost:8000
              predicates:
                - Path=/api/pdf/**
              filters:
                - StripPrefix=2
            - id: code-explainer-service
  
```

```
uri: http://localhost:8001
predicates:
  - Path=/api/code/**
filters:
  - StripPrefix=2
```

Extrait 1 : Configuration des routes dans application.yml

5. Conclusion

Ce chapitre nous a permis de présenter la seconde release de notre application. Le troisième sprint a été consacré au développement du microservice Code Explainer, capable d'analyser automatiquement un dépôt GitHub. Le quatrième sprint a porté sur la mise en place de la Spring Cloud Gateway, point d'entrée unique désormais nécessaire pour accéder aux deux microservices développés. Le chapitre suivant sera consacré à la troisième et dernière release, portant sur le développement de l'interface Angular et la sécurisation complète de l'application via Keycloak.

Chapitre 5 : Release 3 — Frontend et sécurité

1. Introduction

Cette troisième et dernière release vise à rendre notre application utilisable par un utilisateur final, à travers une interface Angular complète, puis à sécuriser l'ensemble du système grâce à Keycloak et à une gestion fine des rôles.

2. Présentation de la release 3

Cette release est décomposée en deux sprints :

- **Sprint 5 : Interface Angular** : Développement de l'interface utilisateur permettant d'exploiter les deux microservices développés dans les releases précédentes.
- **Sprint 6 : Keycloak et gestion des rôles** : Mise en place de l'authentification centralisée et de la distinction entre utilisateurs standards et administrateurs.

3. Sprint 5 : Interface Angular

Ce sprint consiste à développer une interface unique donnant accès aux deux fonctionnalités principales du système, jusqu'ici uniquement testées via la documentation Swagger de chaque microservice.

3.1. Backlog du sprint

ID	User story
US2 à US9	Toutes les fonctionnalités précédemment exposées par API (upload, questions, historique, suppression, analyse GitHub) doivent être accessibles depuis une interface graphique unique et cohérente.

Tableau 13 : Backlog du sprint 5

3.2. Analyse

Ce sprint ne modifie pas le périmètre fonctionnel déjà établi par le diagramme de cas d'utilisation global présenté au chapitre 2 (figure 1) : il s'agit ici de donner une forme graphique aux cas d'utilisation déjà identifiés, sans en introduire de nouveaux.

3.3. Conception

L'application Angular repose sur des composants autonomes (standalone components), organisés comme le montre la figure 14. Le composant racine gère l'état global de connexion via le AuthService ; les deux composants fonctionnels (PdfQa et CodeExplainer) s'appuient chacun sur un service dédié pour communiquer avec la

Gateway. Un intercepteur HTTP unique, commun aux deux services, ajoute automatiquement le token JWT à chaque requête sortante.

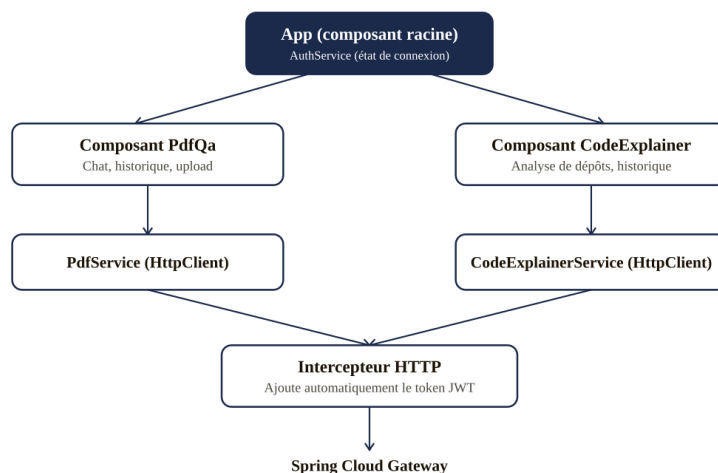


Figure 14 : Architecture interne du frontend Angular

3.4. Réalisation

L'interface développée propose notamment :

- **Chat documentaire** : Un espace de dépôt et de consultation des documents PDF, avec un indicateur clair du document actuellement interrogé.
- **Historique par document** : Un historique de conversation persistant (conservé dans le navigateur), distinct pour chaque document ainsi que pour le mode « Tous les documents ».
- **Analyse de code** : Un champ de saisie d'URL GitHub, avec restitution du résumé, des services et des endpoints détectés.
- **Historique des analyses** : Un historique des dépôts déjà analysés, permettant de revoir un résultat sans relancer une analyse complète.
- **Suppression conditionnelle** : Un bouton de suppression visible uniquement pour les documents dont l'utilisateur est propriétaire, ou pour un administrateur.

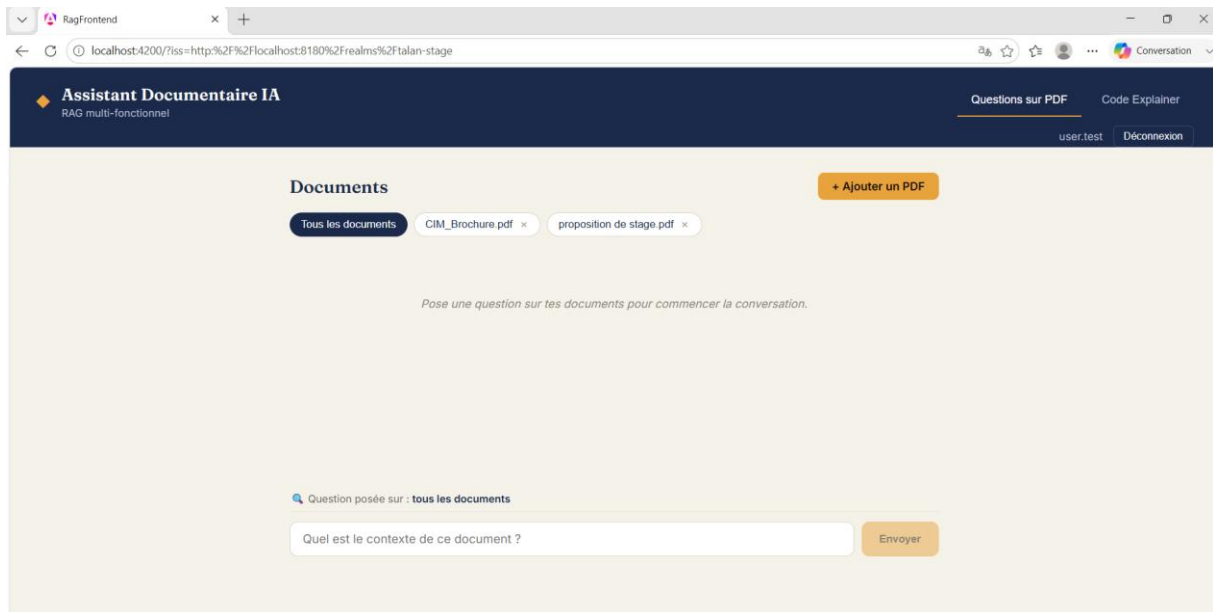


Figure 15 : Interface PDF

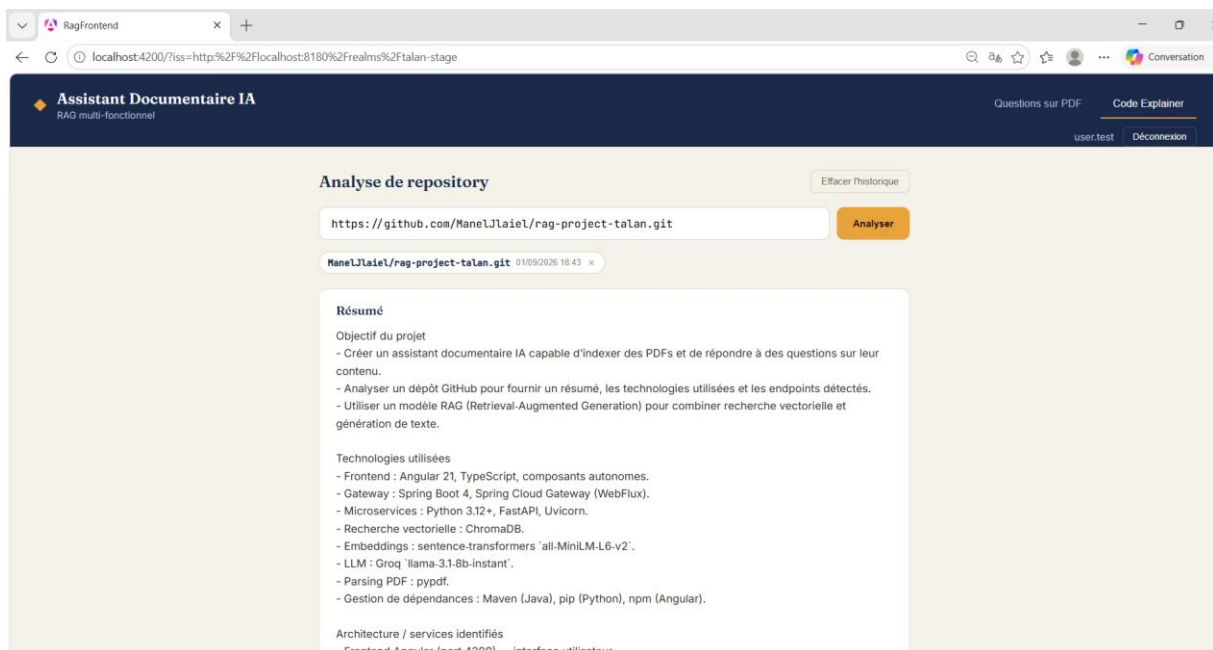


Figure 16 : Interface Code Explainer

4. Sprint 6 : Keycloak et gestion des rôles

Ce dernier sprint vise à sécuriser l'ensemble de l'application : authentification centralisée de tous les utilisateurs, et distinction des droits entre un utilisateur standard et un administrateur.

4.1. Backlog du sprint

ID	User story
US1	En tant qu'utilisateur, je peux m'authentifier à l'application via Keycloak.
US10	En tant qu'administrateur, je peux consulter tous les documents déposés par l'ensemble des utilisateurs.
US11	En tant qu'administrateur, je peux supprimer n'importe quel document, quel que soit son propriétaire.

Tableau 14 : Backlog du sprint 6

4.2. Analyse

Ce sprint s'appuie sur le diagramme de cas d'utilisation global déjà présenté (figure 1, chapitre 2), et plus particulièrement sur la relation de généralisation entre les acteurs Utilisateur et Administrateur, ainsi que sur le cas d'utilisation « Superviser tous les documents », propre à l'administrateur.

4.3. Mécanisme de sécurité

Afin d'assurer une authentification sécurisée et un système d'autorisation robuste, nous avons opté pour Keycloak, solution open source de gestion des identités reposant sur le standard OAuth2 / OpenID Connect. Le flux retenu est l'Authorization Code Flow, considéré comme le plus sûr pour une application cliente exécutée dans un navigateur : le mot de passe de l'utilisateur n'est à aucun moment manipulé par le code Angular, uniquement par Keycloak lui-même.

Une fois authentifié, l'utilisateur reçoit un token JWT (JSON Web Token[10]), constitué d'un en-tête, d'un ensemble de revendications (claims, incluant notamment le nom d'utilisateur et ses rôles) et d'une signature garantissant son intégrité. Ce token est joint à chaque requête envoyée à la Gateway, laquelle vérifie sa signature sans avoir à recontacter Keycloak à chaque appel.

Les rôles définis dans Keycloak (user et admin) sont extraits du token par la Gateway et transmis aux microservices sous forme d'en-têtes HTTP (X-User et X-User-Roles), ces derniers faisant confiance à la Gateway plutôt que de revalider eux-mêmes le token — celle-ci constituant l'unique point d'entrée vers les microservices, qui ne sont pas exposés directement.

4.4. Diagramme de séquence du scénario « S'authentifier »

La figure 17 présente le déroulement complet de l'authentification, depuis la redirection initiale vers Keycloak jusqu'à l'obtention du token JWT.

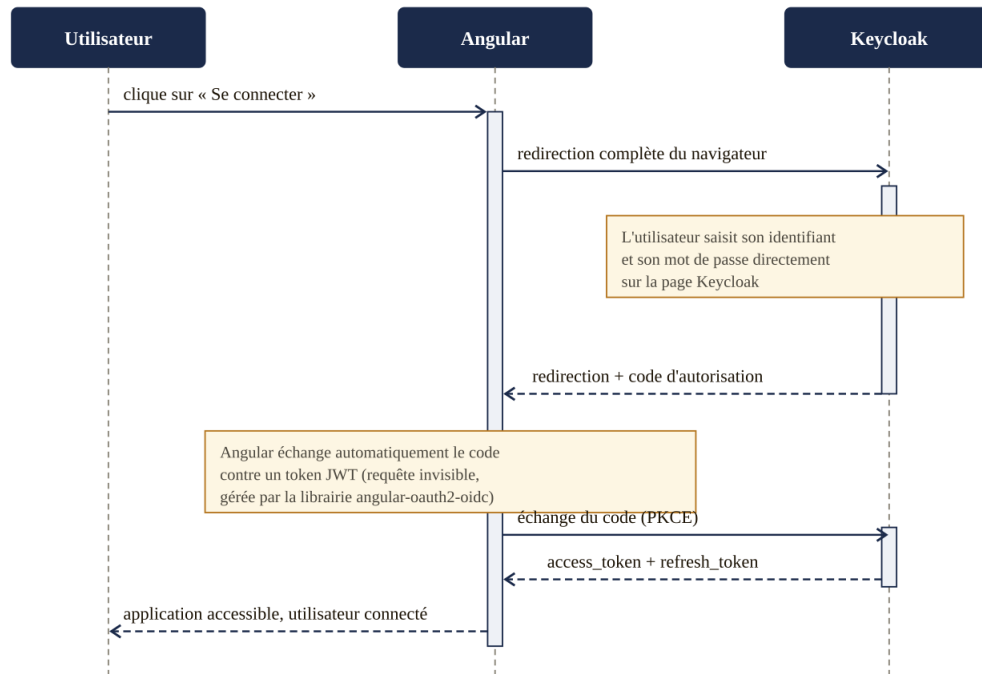


Figure 17 : Diagramme de séquence — authentification via Keycloak

4.5. Réalisation

Trois éléments principaux ont été mis en place pour cette dernière brique :

- **Sécurisation de la Gateway** : La Gateway a été configurée comme OAuth2 Resource Server, avec une classe de sécurité définissant les routes publiques (santé des services) et les routes protégées, ainsi qu'un convertisseur dédié permettant d'interpréter les rôles Keycloak, non reconnus nativement par Spring Security.
- **Écran de connexion** : L'application affiche un écran de connexion tant que l'utilisateur n'est pas authentifié, redirige vers Keycloak, puis restitue l'interface complète une fois la session établie, avec le nom d'utilisateur affiché et un bouton de déconnexion.
- **Interface conditionnée par le rôle** : Le rôle administrateur, décodé directement depuis le token JWT côté client, conditionne l'affichage du bouton de suppression et la visibilité de l'ensemble des documents plutôt que des seuls documents personnels.

Plusieurs difficultés ont été rencontrées et résolues au cours de ce sprint, notamment un conflit entre la configuration CORS et l'activation de Spring Security (nécessitant une configuration

CORS dédiée au niveau de la sécurité elle-même), ainsi qu'un problème d'expiration du token en cours d'utilisation, résolu par l'activation du rafraîchissement automatique proposé par la librairie `angular-oauth2-oidc`.

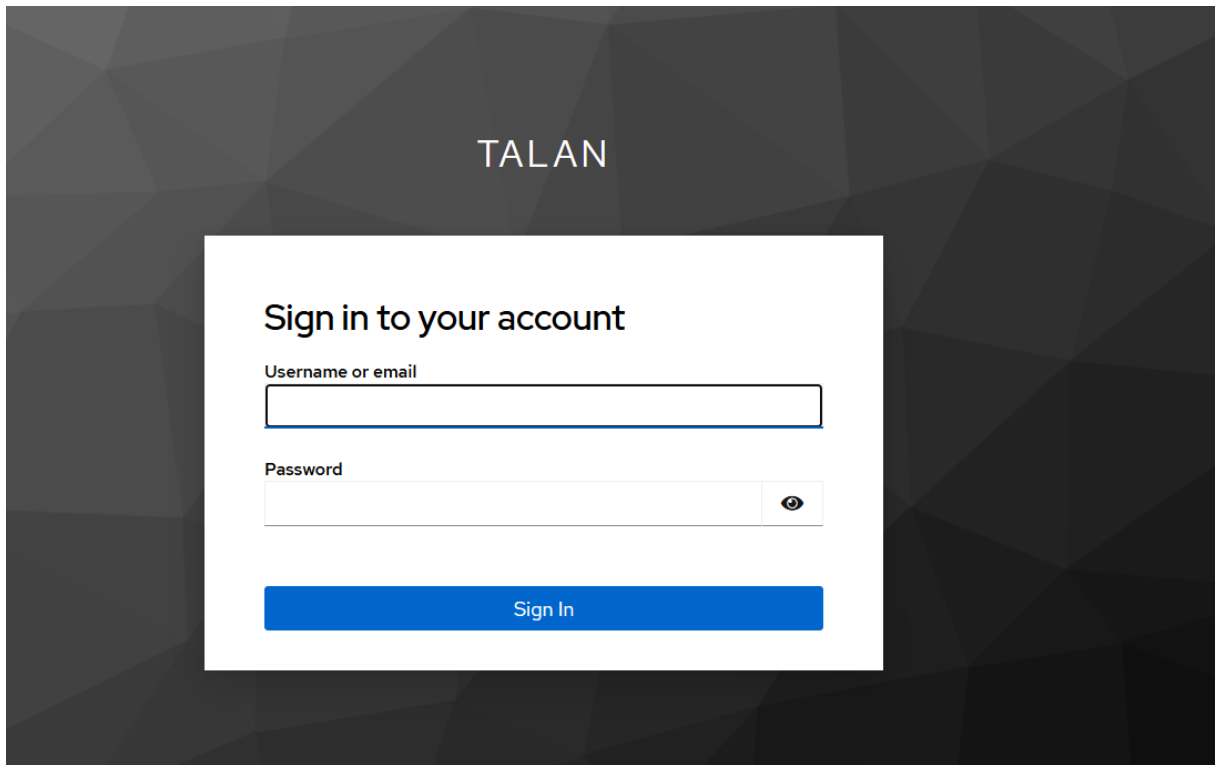


Figure 18 : Interface Login

5. Conclusion

Ce chapitre a permis de rendre notre application pleinement utilisable et sécurisée. Le cinquième sprint a doté le système d'une interface Angular unifiée, exploitant les deux microservices développés dans les releases précédentes. Le sixième et dernier sprint a intégré une authentification centralisée via Keycloak ainsi qu'une gestion des rôles cohérente, distinguant les droits d'un utilisateur standard de ceux d'un administrateur. L'ensemble des fonctionnalités prévues dans le backlog initial est désormais opérationnel.

Conclusion générale

Au terme de ce rapport, nous dressons le bilan du travail réalisé dans le cadre de ce stage de fin d'études, effectué au sein de la société Talan. Ce travail visait à concevoir et développer un assistant documentaire intelligent, capable à la fois d'interroger des documents PDF et d'analyser automatiquement des projets de code hébergés sur GitHub.

Pour mener à bien ce projet, nous avons débuté par une analyse du contexte et des solutions existantes, ce qui nous a permis de dégager une problématique claire et de proposer une solution originale, combinant deux fonctionnalités rarement réunies dans un même outil, au sein d'une architecture sécurisée et intégrée à l'écosystème d'authentification de l'entreprise.

Après avoir fixé les repères de notre application à travers une étude comparative des méthodes agiles, nous avons opté pour une démarche itérative inspirée de SCRUM, adaptée au contexte d'un stage individuel. Cette approche nous a permis de décomposer le projet en trois releases progressives : la première consacrée au moteur RAG et au microservice de gestion des documents PDF, la seconde à l'analyse de code et à la mise en place de la passerelle applicative, et la troisième à l'interface utilisateur ainsi qu'à la sécurisation complète du système.

Ce stage nous a permis de travailler sur un périmètre technique large et cohérent avec les enjeux actuels du développement logiciel : architecture orientée microservices, intelligence artificielle générative et RAG, développement full-stack avec Angular et FastAPI, passerelle applicative avec Spring Cloud Gateway, et sécurité avec Keycloak et OAuth2. Nous avons également été confrontés à des difficultés techniques concrètes tout au long du projet — conflits de versions entre bibliothèques, évolutions de configuration entre versions de frameworks, dépréciation d'un modèle de langage en cours de projet, ou encore ajustements nécessaires à la fiabilité du système RAG — dont la résolution a constitué une part importante et formatrice de ce travail.

L'application développée constitue une base fonctionnelle solide, mais ouvre également plusieurs perspectives d'évolution. Sur le plan fonctionnel, une gestion plus fine des rôles pourrait être envisagée, notamment un espace d'administration dédié à la supervision du système RAG (réindexation, réinitialisation de la base vectorielle) ou à la consultation des sessions actives, en s'appuyant sur l'API d'administration de

Keycloak. Sur le plan technique, une amélioration du RAG par des mécanismes de résumé hiérarchique permettrait de mieux répondre aux questions transversales portant sur un grand nombre de documents simultanément.

Au-delà de l'aspect technique, ce stage a été une opportunité significative d'intégration dans le monde professionnel, au cours de laquelle nous avons pu mettre en pratique, au sein d'une entreprise aux standards technologiques exigeants, des compétences que nous avons continuellement approfondies tout au long de ce travail.

Références bibliographiques

- [1] : <https://fastapi.tiangolo.com/> (Framework FastAPI)
- [2] : <https://angular.dev/> (Framework Angular)
- [3] : <https://spring.io/projects/spring-boot> (Spring Boot)
- [4] : <https://spring.io/projects/spring-cloud-gateway> (Spring Cloud Gateway)
- [5] : <https://www.keycloak.org/documentation> (Keycloak)
- [6] : <https://docs.trychroma.com/> (ChromaDB)
- [7] : <https://console.groq.com/docs> (Groq API)
- [8] : <https://www.sbert.net/> (sentence-transformers)
- [9] : <https://pypdf.readthedocs.io/> (pypdf)
- [10] : <https://jwt.io/introduction> (JSON Web Token)